Email Spam Detection With Naive Bayes, Logistic Regression, and a Neural Network: Accuracy, Weakness to Disguised Spam, and Two Fixes

Alikhan Yessenov, Alua Sarsenova, and Arailym Orynbay

Group BDA-2404

Astana IT University

Applied Machine Learning

Instructor's name: Azim Kassymbayev

November 2026

Abstract

Email spam filters must catch unwanted mail without blocking legitimate messages, while spammers try to evade them. We compared Multinomial Naive Bayes, Logistic Regression and a multilayer perceptron (MLP) on three public corpora (TREC 2005, CEAS 2008 and Enron-Spam). After cleaning and removing duplicates, 122,850 unique emails remained, each represented by 50,000 TF-IDF word and word-pair features. On the held-out test set the MLP was most accurate (F1 0.9866), closely followed by Logistic Regression (F1 0.9845), which trained about 40 times faster; Naive Bayes reached F1 0.9667. We then disguised the test spam. Replacing letters with digits or spacing out spam words had little effect, but appending legitimate-looking words reduced spam recall to 0.195–0.460, and the most accurate models were fooled most. Retraining with disguised spam protected only against the words it was trained on. Of two fixes, scoring each email in 50-word pieces raised recall under attack to 0.603–0.636 at an unchanged false-alarm rate, at the cost of lower recall on normal spam, while training with randomly padded spam gave no protection against new words. We conclude that a regularized linear model offers the best accuracy–cost trade-off and that robustness must come from how emails are scored, not only from more training data.

Keywords: spam detection, text classification, TF-IDF, neural networks, adversarial robustness

Email Spam Detection With Naive Bayes, Logistic Regression, and a Neural Network: Accuracy, Weakness to Disguised Spam, and Two Fixes

Introduction

Unwanted bulk email, or spam, is one of the oldest problems in applied machine learning. Spam wastes the reader's time, fills mailboxes, and is a common way to deliver scams, phishing links and malware. Early filters relied on hand-written keyword rules, but spammers quickly learned to avoid the listed words. Since the late 1990s, most filters have instead been *learned* from labeled examples of spam and legitimate email, which is usually called ham (Sahami et al., 1998; Cormack, 2008).

From a machine learning point of view, spam filtering is a supervised binary text classification problem. Each email is turned into a numeric feature vector, and a classifier estimates the probability that the email is spam. Two kinds of mistakes are possible, and they do not cost the same. A missed spam email is an annoyance, but a legitimate email that is sent to the spam folder (a false positive) can mean a lost job offer, invoice or message from a teacher. A useful filter must therefore keep the false-positive rate on ham very low while still catching most spam.

Spam filtering is also an *adversarial* problem. The people who write spam read the same papers as the people who build filters, and they change their messages to get through. Typical tricks are misspelling trigger words ("fr3e" instead of "free"), splitting them with spaces, and adding blocks of innocent-looking text (Wittel & Wu, 2004; Lowd & Meek, 2005). A filter that scores well on a clean test set may still be easy to fool.

This project, SpamShield, compares three supervised models that represent three families of classifiers: Multinomial Naive Bayes (a probabilistic baseline), Logistic Regression (a discriminative linear model) and a Multilayer Perceptron (a feed-forward neural network). All three use the same TF-IDF features and are trained on more than

120,000 unique emails from three public corpora. The project answers three research questions:

RQ1. Which of the three models classifies email as spam or ham most accurately, with the lowest false-positive rate on ham, and at what computational cost?

RQ2. How robust are the three models when spam is deliberately disguised, for example by changing letters or by adding innocent words?

RQ3. Can the weakness found in RQ2 be reduced, by scoring each email in 50-word pieces or by training with randomly padded spam, without hurting the filter on normal email?

The rest of the paper reviews related work, describes the data and the models, presents the two experiments, and discusses what the results mean for a practical spam filter.

Literature Review

Sahami et al. (1998) were among the first to treat junk-mail filtering as a learning problem. They used a Naive Bayes classifier over word features and showed that it could filter spam with high precision. Androutsopoulos et al. (2000) compared Naive Bayes with keyword-based filtering on personal email and found the learned filter clearly better, but also pointed out that the cost of false positives must be part of the evaluation. Metsis et al. (2006) compared several Naive Bayes variants on the Enron-Spam corpus, which they built for that study, and found that multinomial and flexible variants perform best.

Discriminative linear models soon matched or beat Naive Bayes. Drucker et al. (1999) showed that Support Vector Machines on binary word features were more accurate than boosted trees and the Ripper rule learner for spam categorization. Logistic Regression is closely related to the linear SVM and became a strong standard choice, especially in the online setting of the TREC Spam Tracks (Cormack & Lynam, 2005; Cormack, 2008). Surveys by Guzella and Caminhas (2009) and Cormack (2008) conclude that well-tuned

linear models on word or character features are hard to beat on standard corpora, and that the evaluation method often matters as much as the model.

Neural approaches range from simple multilayer perceptrons on bag-of-words features to recurrent networks and pretrained transformers such as BERT (Devlin et al., 2019). Transformers give the best scores on many text classification benchmarks, but they need far more computation than linear models, and on spam corpora the improvement over a tuned linear model is usually small.

A separate line of work studies attacks on spam filters. Wittel and Wu (2004) described attacks that add random or common words to spam, and Lowd and Meek (2005) showed that a "good word attack", which appends words that are typical of legitimate mail, can make a linear filter miss most spam. Biggio and Roli (2018) review this field of adversarial machine learning more broadly and stress that a classifier should be tested against an attacker, not only on data that looks like its training data.

Three gaps motivate this project. First, many published comparisons use a random split of a single corpus without removing duplicates. Spam campaigns send many identical or near-identical messages, so duplicates can land in both the training and test sets and make scores look better than they are. Second, many studies report accuracy only, while the false-positive rate on ham, which matters most to users, is often missing. Third, robustness to disguised spam is rarely measured side by side for probabilistic, linear and neural models trained on the same features. This project addresses these gaps by deduplicating three corpora before splitting, reporting the false-positive rate together with standard metrics, and testing all three models against the same set of disguise attacks.

Data

Sources

The project combines three public email corpora that are standard in spam research. All three contain real emails with a spam or ham label for every message.

The **TREC 2005 Public Spam Corpus** (Cormack & Lynam, 2005) contains 92,189 raw emails in their original format, including headers, MIME parts and HTML. Its ham messages come from the Enron email collection and its spam was collected in 2005. It was downloaded from the National Institute of Standards and Technology (NIST, n.d.), and its checksum was verified. The **CEAS 2008** corpus contains 39,154 emails collected for the 2008 Conference on Email and Anti-Spam challenge, three years later than TREC 2005. We used the copy in the curated collection described by Champa et al. (2024), obtained through its Hugging Face mirror. The **Enron-Spam** corpus (Metsis et al., 2006) contains 33,716 emails: ham from six Enron employees and spam from several public spam traps. Its text is already lowercased and tokenized.

The project proposal planned to use TREC 2007 instead of CEAS 2008. While preparing the data we found that NIST no longer distributes the TREC 2006 and 2007 corpora because of privacy concerns, so we replaced it with CEAS 2008, which is publicly available and also adds emails from a later year. Together the three corpora contain 165,059 raw emails, well above the 100,000-row requirement. Even after strict deduplication (described below), 122,850 unique emails remain. With only two datasets, the unique count would fall below 100,000, which is the second reason a third corpus was added.

The input feature of every email is its text: the subject line joined with the body. Header fields such as sender, receiver and date are not used. They are missing for part of the data, and they would let a model learn which corpus an email came from instead of what spam looks like. The target is a binary label: 1 for spam and 0 for ham.

Table 1

Number of Emails per Dataset Before and After Cleaning

Dataset	Raw ham	Raw spam	Raw total	Clean ham	Clean spam	Clean total
TREC 2005	39,399	52,790	92,189	31,714	29,127	60,841
CEAS 2008	17,312	21,842	39,154	17,134	18,652	35,786
Enron-Spam	16,545	17,171	33,716	12,460	13,763	26,223
Total	**73,256**	**91,803**	**165,059**	**61,308**	**61,542**	**122,850**

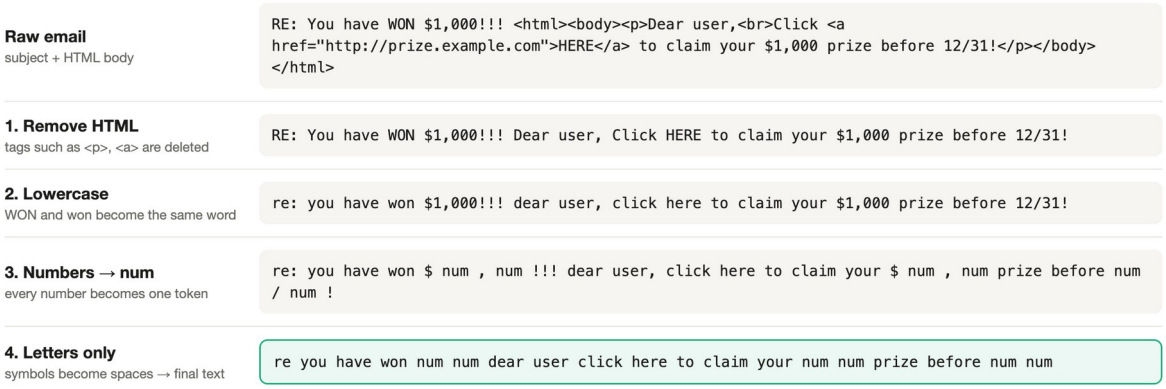
Note. "Clean" counts are after removing empty, conflicting and duplicate emails. Duplicates were removed across all three datasets together.

Preprocessing

Raw TREC 2005 messages were parsed with Python's email package. For each message, the text/plain parts were decoded with their declared character set. When a message had no plain-text part, its text/html parts were used instead. The same cleaning function was then applied to all three corpora so that their different formats became comparable. It removes script and style blocks and HTML tags, decodes HTML entities, and lowercases the text. It replaces every number with the token *num* and every remaining non-letter character with a space. Finally it collapses repeated whitespace and cuts very long emails at 20,000 characters.

Figure 1

One Example Email Before and After Each Cleaning Step



Note. The example email is invented for illustration. The same function is applied to every email in all three datasets.

Replacing numbers with a token and punctuation with spaces was done on purpose. It keeps useful signals such as prices and URLs as words ("num", "http", "www", "com"), and it makes the pre-tokenized Enron-Spam text look the same as the raw TREC and CEAS text. Without it, a model could separate the corpora by formatting artifacts.

Three filters were then applied. Emails with fewer than three words after cleaning were dropped as empty. Texts that appeared with both labels were dropped as contradictory. Exact duplicates of the cleaned text were reduced to one copy. Duplicates were removed across all three corpora *before* the data was split, so no email can appear in both the training and test sets. This step removed 38,816 emails, mostly repeated spam campaigns and Enron emails stored in several folders. The dataset has no missing labels. Missing subjects or bodies were treated as empty strings, and numeric normalization is not needed for text features because TF-IDF vectors are length-normalized.

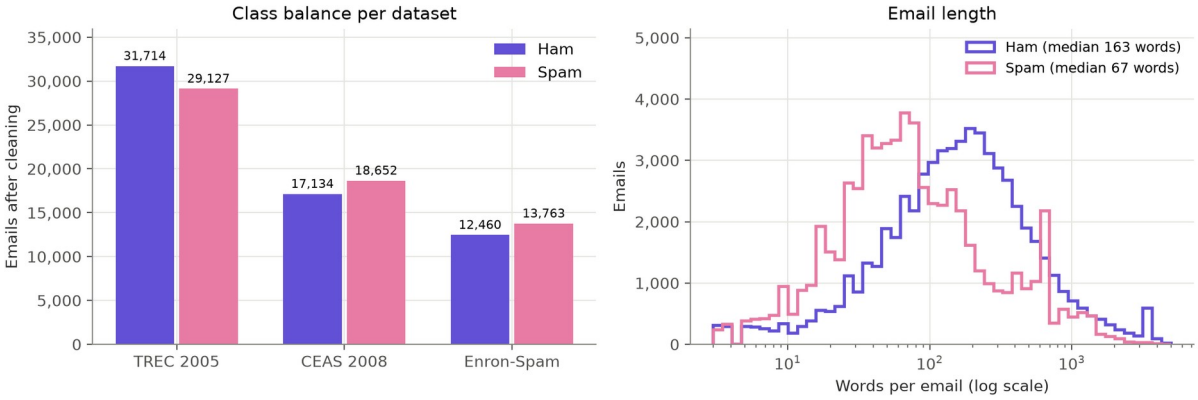
Table 2

Effect of Each Cleaning Step

Step	Emails remaining
Raw emails	165,059
After removing empty / <3 words	161,804
After removing conflicting labels	161,666
After removing duplicates (final)	122,850

Figure 2

Class Balance per Dataset and Distribution of Email Length



Note. Left: number of ham and spam emails after cleaning. Right: number of words per email (logarithmic axis).

Train, Validation, and Test Split

The cleaned data was split into a training set (70%), a validation set (15%) and a test set (15%). The split was stratified by both source corpus and label, so each set has the same mix of corpora and the same share of spam. The random seed was fixed at 42 for reproducibility. The training set was used to fit the TF-IDF vocabulary and the models. The validation set was used to choose hyperparameters and the number of training epochs. The test set was used once, at the end.

Table 3

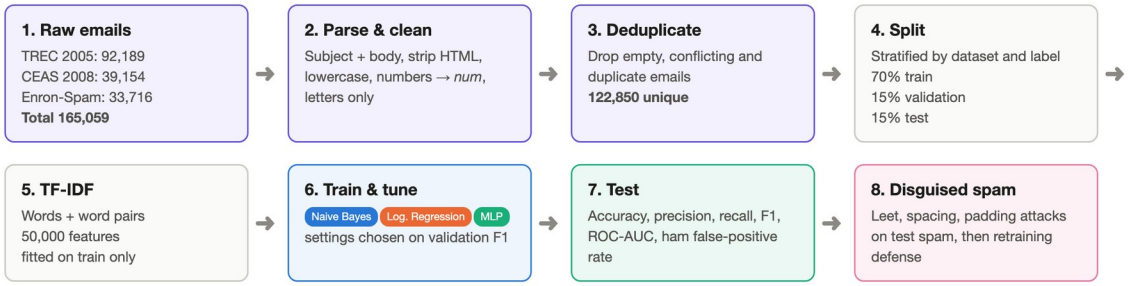
Size of the Training, Validation, and Test Sets

Set	Emails	Share of total	Spam share
Train	85,995	70%	0.501
Validation	18,427	15%	0.501
Test	18,428	15%	0.501

Methodology

Figure 3

SpamShield Data and Modeling Pipeline



Note. Each email passes through steps 1–5 once; the three models in step 6 share the same features.

Emails were converted into TF-IDF vectors (Salton & Buckley, 1988) with scikit-learn (Pedregosa et al., 2011). The vocabulary contains single words and pairs of adjacent words (unigrams and bigrams). It keeps terms that occur in at least 5 training emails and in no more than 90% of them, limited to the 50,000 most frequent terms. Term frequencies are sublinearly scaled (1 + log tf), and every vector is normalized to unit length. The vectorizer was fitted on the training set only, and the same fitted vectorizer transforms the validation, test and disguised emails. All three models therefore receive exactly the same features, so differences between them come from the models and not from the representation. Data handling used pandas (McKinney, 2010), and figures were made with Matplotlib (Hunter, 2007).

Models

Multinomial Naive Bayes

Naive Bayes applies Bayes' theorem under the assumption that words occur independently of each other given the class. The multinomial version models the (TF-IDF-weighted) word counts of each class and predicts the class with the higher posterior probability. It has one hyperparameter, the additive (Laplace/Lidstone) smoothing α, which prevents zero probabilities for words that never appear in one class. Naive Bayes is fast and has a long history in spam filtering (Sahami et al., 1998; Metsis et al., 2006), so it serves as the baseline model.

Logistic Regression

Logistic Regression is a discriminative linear model. It computes a weighted sum of the TF-IDF features and passes it through the sigmoid function to obtain the probability of spam. The weights are learned by minimizing the log loss (binary cross-entropy) with an L2 penalty, using the L-BFGS solver with up to 3,000 iterations. The hyperparameter C is the inverse of the regularization strength: small C gives smaller weights and a simpler model. Each weight corresponds to one word or word pair, so the model is easy to interpret.

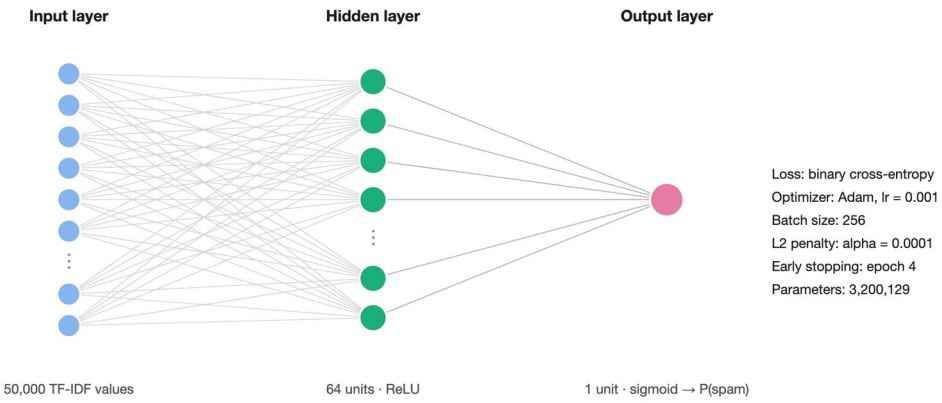
Multilayer Perceptron

The neural model is a feed-forward multilayer perceptron (Figure 4). The 50,000 TF-IDF values form the input layer. They feed one fully connected hidden layer of 64 units with the ReLU activation, followed by a single output unit with a sigmoid activation that gives the probability of spam. The network has 3,200,129 trainable parameters. It is trained with the Adam optimizer (Kingma & Ba, 2015) at a learning rate of 0.001, mini-batches of 256 emails, binary cross-entropy loss and an L2 penalty of $\alpha = 0.0001$. Training runs epoch by epoch on shuffled data. After every epoch the validation loss is computed, and training stops when it has not improved for two epochs (early stopping). The weights from the best epoch

are kept. Unlike the two linear models, the hidden layer lets the MLP learn combinations of words, for example that a word is suspicious only when certain other words are present.

Figure 4

Architecture of the Multilayer Perceptron



Note. Only some of the input and hidden units are drawn.

Training Procedure and Hyperparameter Selection

All hyperparameters were chosen by F1 score on the validation set, not on the test set. A small grid was searched for each model (Table 4). Naive Bayes and Logistic Regression were refitted on the full training set with the selected value. For the MLP, three hidden-layer sizes were compared, each with early stopping, and the size with the best validation F1 was kept. Full 5-fold cross-validation, mentioned in the proposal as optional, was not used. With more than 85,000 training emails, a single large validation set already gives stable estimates, and repeating the MLP training five times per setting would have multiplied the training time.

Table 4

Hyperparameters Searched and Selected

Model	Hyperparameter	Values tried	Selected
Naive Bayes	Smoothing α	0.001, 0.01, 0.1, 0.5, 1	0.001
Logistic Regression	Inverse regularization C	0.1, 1, 10, 100	10
MLP	Hidden units	64, 128, 256	64
MLP	Epochs (early stopping)	up to 12, patience 2	4
MLP	Optimizer / learning rate	Adam / 0.001 (fixed)	—
MLP	Batch size / L2 α	256 / 0.0001 (fixed)	—

Evaluation Metrics

Spam is the positive class. *Accuracy* is the share of correctly classified emails. *Precision* is the share of emails flagged as spam that really are spam, and *recall* is the share of spam that is caught. The *F1 score*, the harmonic mean of precision and recall, is the main metric, as stated in the proposal. Because false positives are the most harmful error, we also report the *false-positive rate (FPR)*: the share of legitimate emails wrongly flagged as spam. *ROC-AUC* and *PR-AUC* (average precision) measure how well each model ranks spam above ham across all thresholds. All metrics use a decision threshold of 0.5. We also report training time and prediction time, because a filter must process every incoming email quickly. A trivial baseline that labels every email as spam is included for reference.

Experiments and Results

Hyperparameter Tuning and Learning Curves

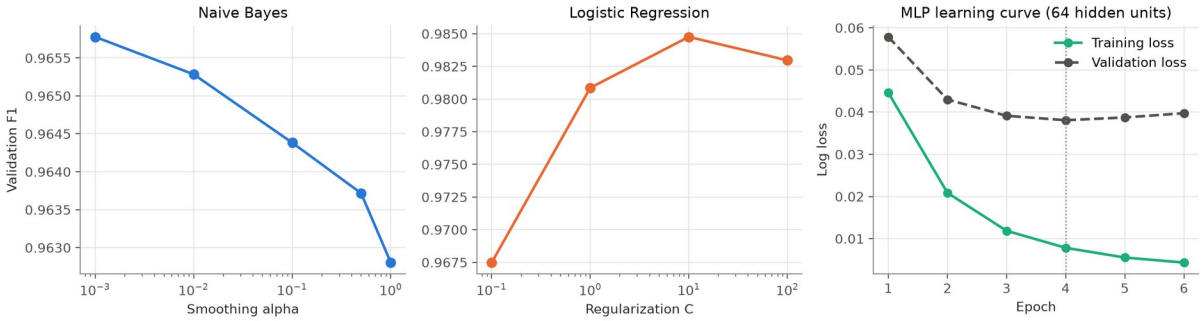
Figure 5 shows how validation F1 changed with each hyperparameter. For Naive Bayes, F1 rose steadily as the smoothing α decreased, from 0.9628 at α = 1 to 0.9658 at α = 0.001. With more than 85,000 training emails, almost every vocabulary term is seen in both classes, so heavy smoothing only blurs real differences between them. The best value is at the edge of the grid, but the curve is flattening and the remaining gain is small. For Logistic Regression, F1 was lowest with strong regularization (C = 0.1, F1 = 0.9675), peaked at C =

10 (F1 = 0.9848) and fell slightly at C = 100. That is the expected pattern: too much regularization underfits, too little starts to overfit.

For the MLP, the hidden-layer size made little difference. The validation F1 at the best epoch was 0.9869 with 64 units, 0.9864 with 128 and 0.9866 with 256, while training time grew from 46 s to 132 s. Larger networks reached their best validation loss earlier (epoch 2 for 256 units against epoch 4 for 64 units), which means they also began to overfit earlier. The smallest network was therefore selected. Its learning curve (Figure 5, right) is a clear case of overfitting being stopped in time. Training loss kept falling to 0.0043, while validation loss was lowest at epoch 4 (0.0381) and then rose again. Early stopping kept the weights from epoch 4.

Figure 5

Validation F1 for Each Setting and MLP Learning Curve



Note. Left and centre: validation F1 against the hyperparameter (logarithmic axes). Right: training and validation log loss per epoch for the selected MLP; the dotted line marks the epoch kept by early stopping.

Test-Set Performance (RQ1)

Table 5 reports the final results on the 18,428 test emails. All three models are far better than the trivial baseline, which labels every email as spam and reaches only 0.501 accuracy because the cleaned data is almost perfectly balanced. The MLP achieved the best accuracy (0.9866), recall (0.9885), F1 (0.9866) and ROC-AUC (0.9989). Logistic Regression was a close second (F1 = 0.9845), and Naive Bayes was clearly third (F1 = 0.9667). The gap between the MLP and Logistic Regression is only 0.21 percentage points of F1, or about 40

fewer errors on 18,428 emails. The gap between Logistic Regression and Naive Bayes is about 1.8 points.

The ranking is different for false positives. Naive Bayes had the highest precision (0.9870) and the lowest false-positive rate: it flagged 1.25% of legitimate emails, compared with 1.53% for the MLP and 1.86% for Logistic Regression. It paid for this with much lower recall, missing 5.3% of spam. All three models met every success criterion in the project proposal: F1, precision and accuracy of at least 0.95, and recall of at least 0.93 on the test set.

The costs differ by orders of magnitude. Naive Bayes trained in 0.06 seconds and Logistic Regression in 1.2 seconds. The MLP needed 46 seconds, including the per-epoch validation checks, about 40 times longer than Logistic Regression. Prediction was fast for all three: under 2 ms per 1,000 emails. This is negligible for a real mail server.

Table 5

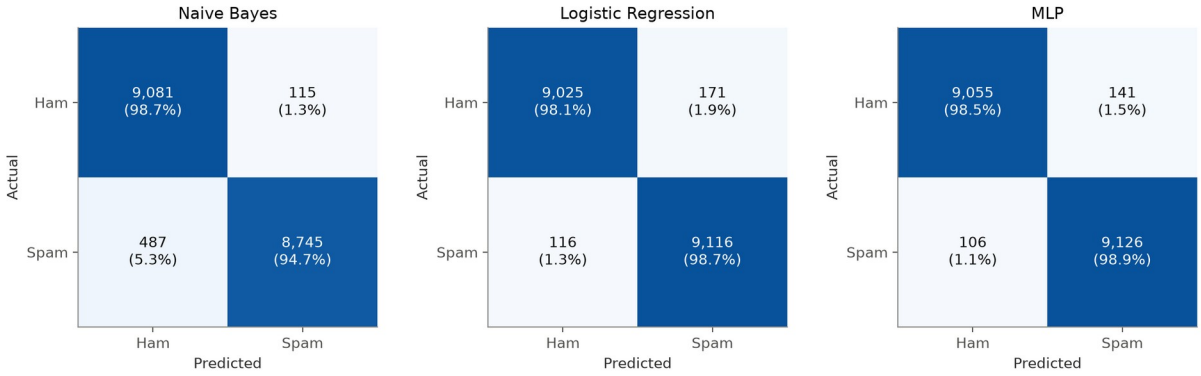
Test-Set Results of the Three Models

Model	Acc.	Prec.	Recall	F1	ROC-AUC	FPR	Train (s)	ms / 1k
Naive Bayes	.9673	.9870	.9472	.9667	.9965	1.25%	0.06	1.0
Log. Regression	.9844	.9816	.9874	.9845	.9985	1.86%	1.16	0.2
MLP	**.9866**	**.9848**	**.9885**	**.9866**	**.9989**	**1.53%**	**46.48**	**1.8**
Always spam	.5010	.5010	1.000	.6675	.5000	100%	0.00	0.0

Note. Test set of 18,428 emails; threshold 0.5. Precision, recall and F1 refer to the spam class. FPR = share of ham emails flagged as spam. Train (s) = wall-clock fitting time on an Apple M3 Pro CPU (for the MLP, including per-epoch validation checks). ms / 1k = prediction time per 1,000 emails.

Figure 6

Confusion Matrices on the Test Set



Note. Rows are true classes and columns are predicted classes; percentages are shares of each true class.

The confusion matrices (Figure 6) show where the errors fall. Naive Bayes made the fewest false positives (115) but by far the most false negatives (487 missed spam emails). Logistic Regression and the MLP both missed far less spam (116 and 106), at the price of more false positives (171 and 141). The ROC curves in Figure 7 use a logarithmic false-positive axis because that is the region that matters for a filter. Below a false-positive rate of 1%, the MLP and Logistic Regression catch clearly more spam than Naive Bayes at the same rate of false alarms.

A false-positive rate of 1–2% at the default threshold of 0.5 is still too high for a real mailbox: one legitimate email in fifty or a hundred would go to the spam folder. Because the models output probabilities, the threshold can be raised. Table 6 shows the effect of choosing the threshold on the validation ham so that only 1%, 0.5% or 0.1% of it would be flagged, and then applying that threshold to the test set.

Table 6

Test Results When the Threshold Is Chosen for a Target False-Positive Rate

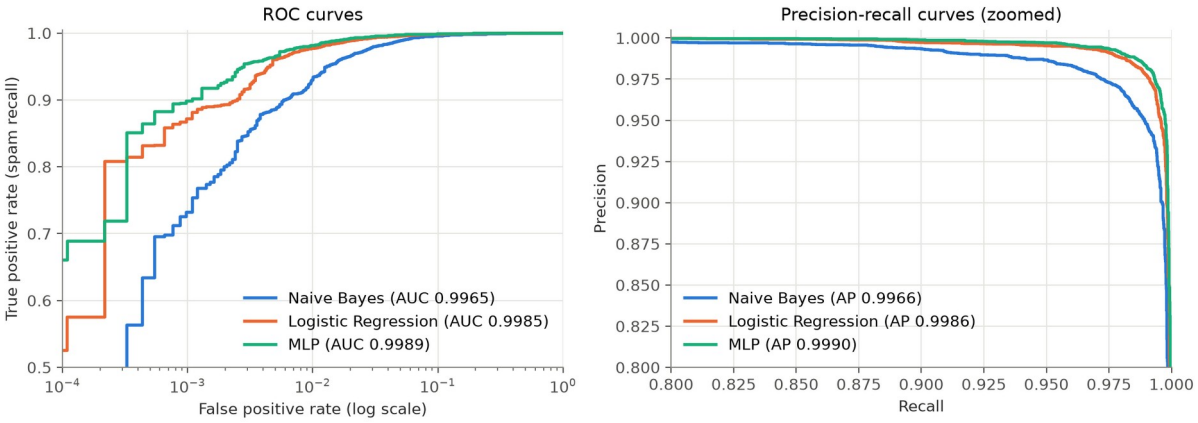
Model	Target FPR	Threshold	Test FPR	Test recall	Test F1
Naive Bayes	1%	0.498	1.25%	0.9476	0.9669
Naive Bayes	0.5%	0.746	0.54%	0.8929	0.9407
Naive Bayes	0.1%	0.984	0.11%	0.7421	0.8514
Logistic Regression	1%	0.700	0.91%	0.9758	0.9832
Logistic Regression	0.5%	0.841	0.46%	0.9511	0.9727
Logistic Regression	0.1%	0.968	0.07%	0.8450	0.9157
MLP	1%	0.670	1.12%	0.9830	0.9859
MLP	0.5%	0.872	0.48%	0.9630	0.9788
MLP	0.1%	0.988	0.05%	0.8803	0.9361

Note. Each threshold was chosen on the validation ham only (the corresponding quantile of predicted spam probabilities) and then applied unchanged to the test set.

With a threshold that targets a 0.5% false-positive rate, the MLP still catches 96.3% of spam, Logistic Regression 95.1% and Naive Bayes only 89.3%. At a 0.1% target the differences grow further: MLP 88.0%, Logistic Regression 84.5% and Naive Bayes 74.2%. The low false-positive rate of Naive Bayes at the default threshold is therefore not a real advantage. Its probabilities are poorly ranked in the region that matters, and once all models are tuned to the same false-alarm rate, the MLP and Logistic Regression are clearly safer and more useful. The test false-positive rates stay close to the targets chosen on validation data, which shows that the thresholds transfer reliably.

Figure 7

ROC and Precision–Recall Curves on the Test Set



Note. The ROC curve uses a logarithmic false-positive axis to show the low-FPR region that matters for spam filtering; the precision–recall panel is zoomed to the upper-right corner.

Results by Source Dataset

Table 7 and Figure 8 break the test results down by source corpus. All three models found CEAS 2008 the easiest (MLP F1 = 0.9963) and TREC 2005 the hardest (MLP F1 = 0.9790). TREC 2005 also had the highest false-positive rates, up to 2.5% for Logistic Regression. Its ham comes from thousands of Enron employees and mixes business, personal and newsletter mail, so it is more varied than the curated Enron-Spam folders or the CEAS mailing lists. The advantage of Logistic Regression and the MLP over Naive Bayes was largest on Enron-Spam and TREC 2005, where Naive Bayes missed about 6.9% and 7.6% of spam.

Table 7

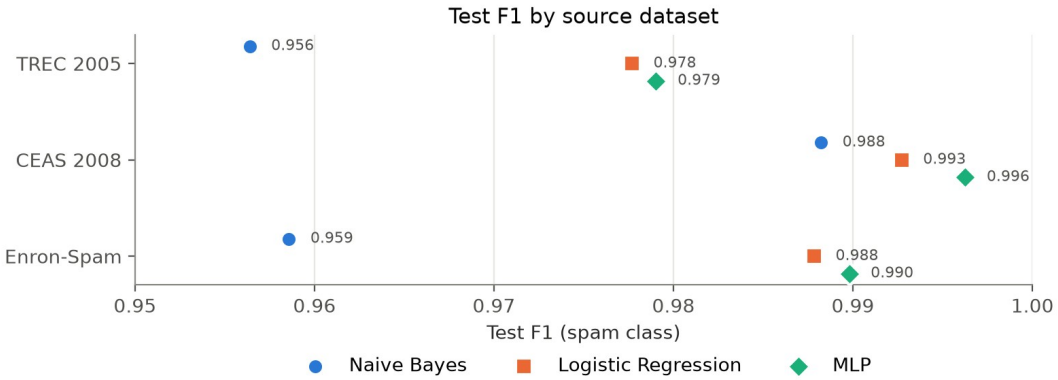
Test F1 and False-Positive Rate by Source Dataset

Dataset	F1 NB	F1 LR	F1 MLP	FPR NB	FPR LR	FPR MLP
TREC 2005	.9564	.9777	.9790	1.47%	2.54%	2.33%
CEAS 2008	.9882	.9927	.9963	1.40%	1.36%	0.66%
Enron-Spam	.9586	.9878	.9898	0.48%	0.80%	0.70%

Note. NB = Naive Bayes; LR = Logistic Regression; MLP = multilayer perceptron.

Figure 8

Test F1 by Source Dataset



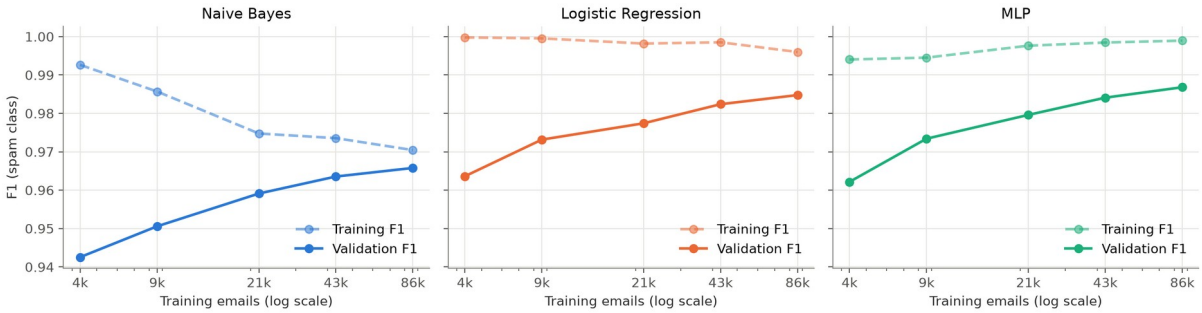
Note. The horizontal axis starts at 0.95.

Overfitting, Underfitting, and Training-Set Size

Figure 9 shows training and validation F1 as each model is retrained on 5% to 100% of the training set. The curves show two different behaviors. Naive Bayes *underfits*. Its training F1 falls from 0.993 to 0.970 as more data is added, its validation F1 rises only to 0.966, and the two curves almost meet. The model is too simple to fit the data well, and more data would not help much. Logistic Regression and the MLP fit the training data almost perfectly (training F1 above 0.99 at every size), so they have the opposite problem: a gap between training and validation scores, which is a mild form of *overfitting*. That gap shrinks steadily as data is added. Validation F1 rises from 0.964 to 0.985 for Logistic Regression and from 0.962 to 0.987 for the MLP and is still increasing at full size, which suggests that more training data would improve both models further. With fewer than 10,000 training emails, Logistic Regression matched the MLP. The MLP pulled ahead only when it had more data, which is typical for models with many more parameters.

Figure 9

Training and Validation F1 as the Training Set Grows



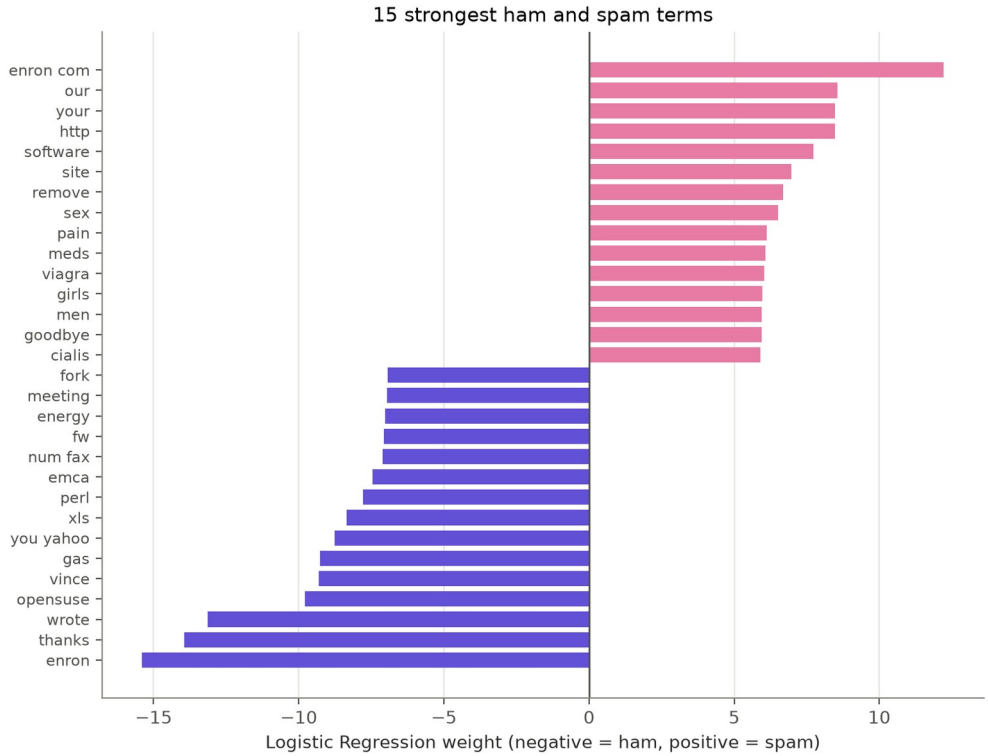
Note. Each model was retrained on random subsets of 5%, 10%, 25%, 50%, and 100% of the training set; the horizontal axis is logarithmic.

What the Models Learned

Figure 10 shows the words with the largest Logistic Regression weights. Many spam terms are what one would expect: product names (*viagra*, *cialis*, *meds*), adult content (*sex*, *girls*), and marketing language (*your*, *our*, *remove*, *site*, *software*, *http*). The ham side, however, shows the main weakness of public corpora. The strongest ham terms include *enron*, *vince* (an Enron employee), *gas* and *energy* (Enron's business), *opensuse*, *perl* and *fork* (open-source mailing lists in CEAS 2008), *wrote* and *thanks* (typical of replies). The strongest spam term of all is the bigram *enron com*, which appears in spam that was sent to Enron addresses. These are real patterns in the data, but they describe *who received the email* more than *what spam looks like*. A filter trained on these corpora would rely on such words, which limits how well it transfers to a different organization. The disguised-spam experiment below shows how an attacker can exploit them.

Figure 10

The 15 Strongest Ham and Spam Terms in the Logistic Regression Model



Note. Negative weights push an email toward ham, positive weights toward spam. Bigrams are shown with a space.

Errors are not spread evenly across emails. For the MLP, 94.1% of emails shorter than 20 words were classified correctly, compared with about 99% for every longer length group. The false-positive rate in that group was 4.1%, roughly ten times higher than for medium-length emails. Very short emails, such as one-line replies or forwarded links, give the model few words to judge by, so these are the legitimate emails most at risk of being lost.

Experiment 2: Disguised Spam (RQ2)

The second experiment tests RQ2. The trained models and the fitted TF-IDF vectorizer were frozen, and only the spam emails of the test set were modified. Ham emails were left unchanged, so any drop in performance shows up as lower spam recall. The disguised text was passed through the same cleaning function as real email, as a deployed filter would do. Five attacks were tested:

- **Leet:** about half of the replaceable letters in each targeted spam word are swapped for digits (*viagra → v1agr4*). The cleaning step then turns each digit into the token *num*.
- **Spacing:** the letters of each targeted spam word are separated by spaces (*viagra → v i a g r a*), which turns the word into one-letter tokens that TF-IDF ignores.
- **Generic padding:** 30 words, drawn at random from a list of 20 everyday office words (*meeting*, *project*, *thanks*, *schedule*, …), are appended to the end of each spam email.
- **Informed padding:** 30 words are appended, drawn from the 30 words most typical of ham in the training data. This simulates an attacker who knows the victim's mail, the "good word attack" of Lowd and Meek (2005).
- **Combined:** each targeted spam word is randomly leet-coded or spaced, and informed padding is added.

The targeted spam words were chosen from the training data only. They are the 50 words that occur in at least 3% of training spam and are most over-represented in spam compared with ham, such as *rolex*, *viagra*, *pills*, *watches*, *prescription*, *replica*, *sex*, *thousand*. 66.6% of the test spam emails contain at least one of these words, so the leet and spacing attacks change only those emails. The ham words used for informed padding are, for example, *tangomu*, *opensuse*, *dev*, *kaminski*, *python*, *louise*, *fyi*, *wrote*.

Table 8

Spam Recall on Disguised Test Spam

Attack	All: NB	All: LR	All: MLP	Targeted: NB	Targeted: LR	Targeted: MLP
Original	.9472	.9874	.9885	.9716	.9928	.9940
Leet	.9448	.9857	.9883	.9678	.9902	.9937
Spacing	.9403	.9842	.9869	.9612	.9880	.9915
Generic padding	.7022	.3365	.5552	.8414	.4699	.7216
Informed padding	.4874	.2283	.3764	.6329	.3205	.5116
Combined	.4599	.1954	.3267	.5913	.2698	.4382

Note. "All" = recall over all 9,232 test spam emails. "Targeted" = recall over the 6,153 test spam emails that contain at least one targeted spam word (the only emails the leet and spacing attacks change). Ham emails are unchanged. All values are recall, not F1; F1 under attack is given in the text. NB = Naive Bayes; LR = Logistic Regression.

Figure 11

Spam Recall of Each Model Under Each Attack

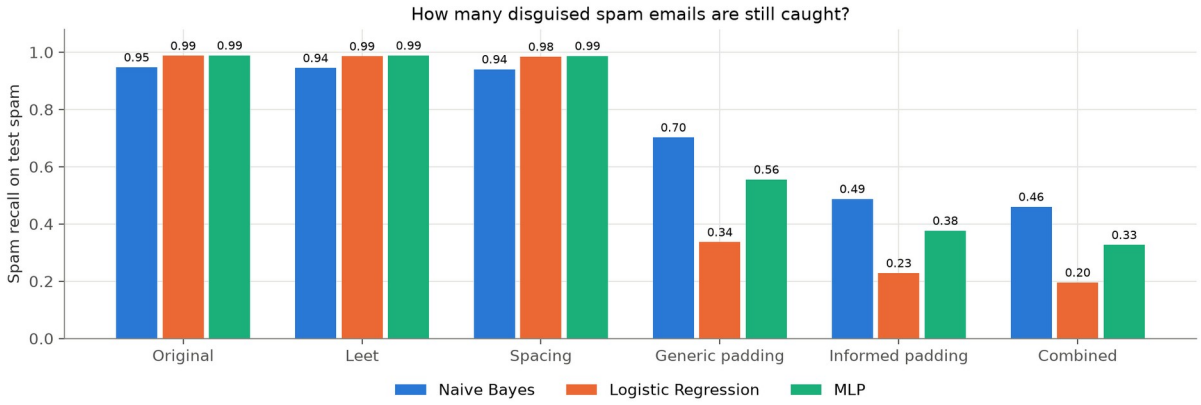


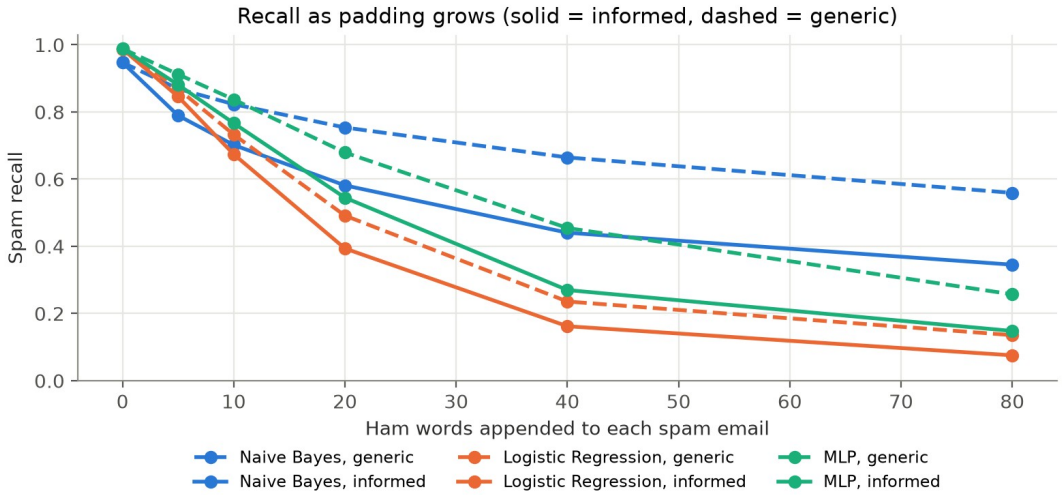
Table 8 and Figure 11 show the results. The two character-level attacks hurt the models much less than expected. Leet-coding lowered recall to 0.945 for Naive Bayes, 0.986 for Logistic Regression and 0.988 for the MLP, and spacing to 0.940, 0.984 and 0.987. Even among the spam emails that actually contained targeted words, at least 96.1% were still caught by every model. Two things explain this. First, spam is redundant: hiding the 50 most typical spam words still leaves URLs, marketing phrases and many other signals. Second, the disguises leave traces of their own. The digits in leet words become *num* tokens, which are common in spam, and spaced-out words leave behind an unusual pattern of word fragments.

Padding was far more effective. Appending 30 generic office words reduced recall to 0.702 for Naive Bayes, 0.337 for Logistic Regression and 0.555 for the MLP. Informed padding was worse for every model: 0.487, 0.228 and 0.376 respectively. The combined attack gave 0.460, 0.195 and 0.327. In other words, a few dozen innocent words at the end of an email let most spam through every model. Because legitimate emails are not changed by the attack, F1 under attack follows directly from this recall and the unchanged false-positive rate: under the combined attack, F1 fell from 0.9667 to 0.625 for Naive Bayes, from 0.9845 to 0.322 for Logistic Regression and from 0.9866 to 0.487 for the MLP.

The ranking under attack is the reverse of the ranking on clean data. Naive Bayes, the least accurate model on clean email, kept the most spam under every padding attack, and Logistic Regression, the second most accurate, kept the least. The weights in Figure 10 suggest why. With weak regularization (C = 10), Logistic Regression gives a few ham words very large negative weights, so a handful of them is enough to cancel the evidence from many spam words. Naive Bayes, by contrast, spreads its evidence over every word through smoothed log-probabilities, so its decision depends less on any small group of words. The MLP sits between the two: its hidden layer mixes evidence from many words, but it is trained on the same features and learns similar shortcuts. Because every TF-IDF vector is normalized to unit length, appended words also lower the relative weight of the original spam words, which is why even generic office vocabulary works as an attack.

Figure 12

Spam Recall as the Number of Appended Ham Words Grows



Note. Solid lines: informed padding (most ham-like training words). Dashed lines: generic office words.

Figure 12 shows how recall falls as more words are appended. The decline is steep and continuous. Even 10 informed words lowered recall to 0.702 for Naive Bayes, 0.674 for Logistic Regression and 0.766 for the MLP. With 80 words, no model caught more than 55.9% of spam. Informed words were more damaging than generic office words for all three models, but the generic words alone were enough to defeat Logistic Regression and the MLP: with 40 generic words their recall fell to 0.236 and 0.455. An attacker therefore does not need inside knowledge of the victim's mail. A long block of ordinary business vocabulary is enough.

First Attempt at a Fix: Retraining With the Attack's Own Words

A natural first idea is to show the models disguised spam during training. Each model was retrained on the original training set plus one disguised copy of every training spam email, made with a randomly chosen attack (leet, spacing, informed padding or combined). These copies used the same 30 informed words as the test attack, so this is the best case for the defender: the attacker's words are known in advance. Generic padding was left out of

training to see what happens with words the model has not seen. New disguised test spam was generated with a different random seed. The results are in Table 9 and Figure 13.

Retraining worked very well against the attacks the models had seen. Under the combined attack, recall rose from 0.460 to 0.990 for Naive Bayes, from 0.195 to 0.999 for Logistic Regression and from 0.331 to 0.998 for the MLP, while results on the clean test set barely changed. However, recall under *generic* padding, whose words were not in the training copies, improved little or not at all: from 0.699 to 0.739 for Naive Bayes, from 0.337 to 0.331 for Logistic Regression and from 0.554 to 0.548 for the MLP. The models learned to distrust the specific words they were shown, not the general idea of padding. An attacker who simply switches to other innocent words would get through again. Experiment 3 looks for fixes that work no matter which words the attacker adds.

Table 9

Effect of Retraining With the Attack's Own Word Lists (First Attempt)

Model	Clean F1	Clean FPR	Leet	Spacing	Generic pad.	Informed pad.	Comb.
Naive Bayes (original)	.9667	1.25%	.9450	.9403	.6988	.4910	.4597
Naive Bayes (retrained)	.9694	1.65%	.9567	.9494	.7393	.9901	.9899
Log. Regression (original)	.9845	1.86%	.9852	.9842	.3367	.2276	.1950
Log. Regression (retrained)	.9838	2.19%	.9899	.9891	.3315	.9988	.9988
MLP (original)	.9866	1.53%	.9881	.9869	.5543	.3741	.3307
MLP (retrained)	.9859	1.53%	.9884	.9866	.5479	.9974	.9976

Note. Attack columns show spam recall on disguised test spam generated with a different random seed than the training copies.

Figure 13

Spam Recall Under Attack Before and After the First Retraining Attempt



Experiment 3: Two Fixes for the Padding Weakness (RQ3)

Experiment 2 showed the weakness and why it happens. A bag-of-words model gives every word a weight and adds the weights up over the whole email. A block of ham-typical words therefore outweighs the spam words, and because every email vector is scaled to the same length, the added words also shrink the weight of the real spam words. The first retraining attempt protected only against the exact words it was trained on. We therefore tested two fixes that are meant to work for *any* added words. Both reuse the same TF-IDF features and the same hyperparameters as before.

Fix 1: Scoring the Email in 50-Word Pieces

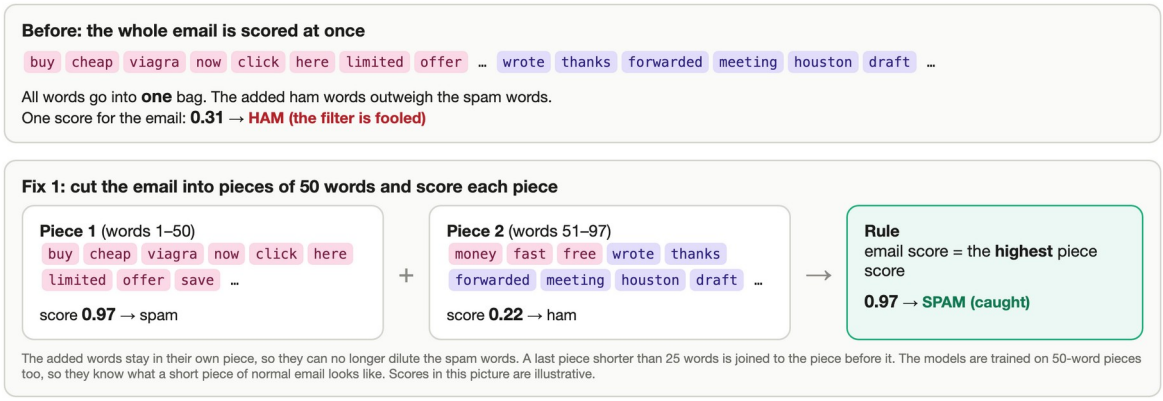
The first fix changes how an email is scored, not which words the model knows. Each cleaned email is cut into consecutive pieces of 50 words. If the last piece has fewer than 25 words, it is joined to the piece before it, so that no piece is too short to judge. Every piece is scored separately, and the email's spam score is the *highest* score of any of its pieces (Figure 14). Words added to the end of a spam email therefore end up mostly in a piece of their own, while the piece that holds the original spam text still looks like spam.

Models trained on whole emails are not used to judging short pieces of text. In a preliminary trial, applying the original models to 50-word pieces flagged far more legitimate emails, because some short part of a normal email, such as a signature or a list of links, often looked suspicious on its own. The three models were therefore retrained on 50-word pieces of

the training emails, with each piece taking the label of its email. This gave 429,462 training pieces instead of 85,995 emails. The hyperparameters stayed the same (α = 0.001, C = 10, 64 hidden units, 4 epochs).

Figure 14

How Fix 1 Scores an Email



Note. Pink words are spam-typical, purple words are ham-typical. The scores are illustrative.

Fix 2: Training With Randomly Padded Spam

The second fix changes the training data instead of the scoring. For every spam email in the training set, one extra copy was made with between 10 and 80 words added at the end (Figure 15). The added words were drawn at random from a pool of strongly ham-typical training words: the 500 words, among those appearing in at least 0.2% of legitimate training emails, that are most over-represented in ham. Every word used by any test attack (the 30 informed-padding words and the 20 generic office words) was removed from the pool, which left 467 words, for example *zvllln*, *estoppel*, *ees*, *counterparty*, *smsbmo*, *ena*, *mmbtu*, *enforceable*. Because the test words are new to the model, the test shows whether a model learns the *trick* (spam with extra ham words is still spam) or only memorizes the words it saw. The models were retrained on the original training emails plus the 43,079 padded copies, 129,074 rows in total.

A third version, Fix 1 + 2, combines both ideas: the models are trained on 50-word pieces of both the original training emails and the padded copies (618,042 pieces), and an email is scored by its highest piece.

Figure 15

How Fix 2 Creates Extra Training Examples



Note. The padding words in this picture are examples of ham-typical words; in the experiment none of the words used in the tests were in the training pool.

How the Fixes Were Compared

All versions were tested on the same test set as before. A filter can catch more spam simply by flagging more email, so a fair comparison must hold the false-alarm rate constant. For every fixed version, the decision threshold was therefore chosen on the validation set so that it flags the same share of legitimate validation email as the original model does at the usual threshold of 0.5. Three things are then compared on the test set: spam recall on normal, undisguised spam (the cost of the fix), the false-positive rate on legitimate email (which should stay the same), and spam recall under the three padding attacks from Experiment 2 (the benefit of the fix). The disguised test spam was generated with a new random seed.

Results of the Fixes

Table 10

Results of the Two Fixes on the Test Set

Model / version	Clean F1	Clean recall	FPR	Gen. pad.	Inf. pad.	Comb. recall	Comb. F1
Naive Bayes, original	0.9667	0.9472	1.25%	0.7013	0.4904	0.4606	0.6253
Naive Bayes, Fix 1	0.8642	0.7682	0.97%	0.6603	0.6034	0.6027	0.7476
Naive Bayes, Fix 2	0.9521	0.9188	1.13%	0.6445	0.4457	0.4144	0.5814
Naive Bayes, Fix 1 + 2	0.8626	0.7665	1.07%	0.6521	0.5964	0.5974	0.7430
Log. Regression, original	0.9845	0.9874	1.86%	0.3369	0.2283	0.1943	0.3204
Log. Regression, Fix 1	0.9358	0.8955	1.85%	0.6065	0.5939	0.6360	0.7689
Log. Regression, Fix 2	0.9828	0.9847	1.94%	0.2971	0.1896	0.1573	0.2674
Log. Regression, Fix 1 + 2	0.9163	0.8617	1.92%	0.5852	0.5803	0.6172	0.7544
MLP, original	0.9866	0.9885	1.53%	0.5576	0.3739	0.3297	0.4903
MLP, Fix 1	0.9223	0.8686	1.51%	0.6158	0.5838	0.6218	0.7597
MLP, Fix 2	0.9860	0.9872	1.52%	0.4949	0.2933	0.2647	0.4137
MLP, Fix 1 + 2	0.9119	0.8530	1.78%	0.6182	0.5836	0.6172	0.7550

Note. Clean = original test emails without any attack. FPR = share of legitimate test emails flagged as spam; each fixed version's threshold was chosen on the validation set to match the original model's false-positive rate. The three padding columns show spam recall (share of disguised test spam still caught). Comb. F1 = F1 on a test set in which every spam email carries the combined attack; it follows from the recall and the false-positive rate because legitimate emails are not changed. NB = Naive Bayes; LR = Logistic Regression.

Table 10 shows the results for all twelve combinations of model and version. As intended, every version flags about the same share of legitimate test emails as its original model (false-positive rates between 0.97% and 1.94%), so the versions can be compared directly.

Fix 1 made Logistic Regression and the MLP much harder to fool, and Naive Bayes somewhat harder. Under the combined attack, spam recall rose from 0.461 to 0.603 for Naive Bayes, from 0.194 to 0.636 for Logistic Regression (3.3 times higher) and from 0.330 to 0.622 for the MLP. For Logistic Regression and the MLP, generic and informed padding showed the same pattern (Figure 16). For Naive Bayes, which was already the most robust model, Fix 1 helped against informed padding (0.490 to 0.603) but not against generic padding (0.701 to 0.660). The fix works whatever words the attacker chooses, because it does

not depend on the words at all: it only stops them from being mixed with the spam text. In terms of F1 under the combined attack, Fix 1 raised the score from 0.625 to 0.748 for Naive Bayes, from 0.320 to 0.769 for Logistic Regression and from 0.490 to 0.760 for the MLP.

Fix 1 has a clear price. On normal, undisguised spam, recall fell from 0.947 to 0.768 for Naive Bayes, from 0.987 to 0.895 for Logistic Regression and from 0.989 to 0.869 for the MLP, so F1 on the clean test set dropped to 0.864, 0.936 and 0.922. A 50-word piece holds less evidence than a whole email, and a long legitimate email has several pieces, each with its own chance of looking suspicious. To keep false alarms at the original level, the threshold had to rise to between 0.91 and 0.99, and spam whose pieces are all only moderately suspicious now falls below it.

Figure 16

Spam Recall of Each Version Without Attack and Under Each Padding Attack



Note. All versions of a model flag about the same share of legitimate email, so the bars can be compared directly.

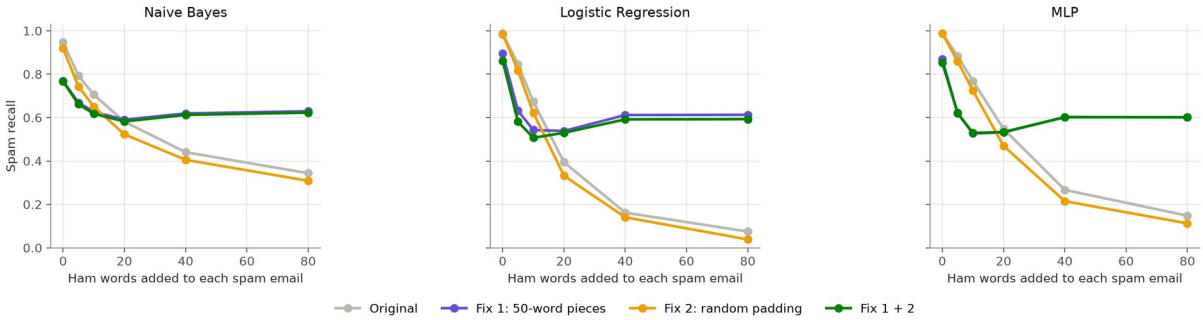
Fix 2 did not help and was even slightly worse. Under the combined attack, recall changed from 0.461 to 0.414 for Naive Bayes, from 0.194 to 0.157 for Logistic Regression and from 0.330 to 0.265 for the MLP. Generic and informed padding showed the same picture, while results on clean email stayed almost unchanged (F1 0.952, 0.983 and 0.986). The reason lies in how these models store what they learn: every word has its own weight. Training with padded copies taught the models to give less weight to the 467 pool words, but the attack words were not in the pool, so their weights stayed as strong as before. Together

with the first attempt, this gives a clear lesson: retraining with padded spam protects only against the words it has seen.

Combining both fixes gave almost the same result as Fix 1 alone. Under the combined attack, Fix 1 + 2 reached 0.597, 0.617 and 0.617, against 0.603, 0.636 and 0.622 for Fix 1, so the training part of Fix 2 added little on top of piece scoring.

Figure 17

Spam Recall as More Ham Words Are Added, Before and After the Fixes



Note. Informed padding with 0 to 80 added words. Thresholds are the same as in Table 10.

Figure 17 shows the difference most clearly. For the original models, recall keeps falling as more words are added, down to 0.075–0.344 with 80 added words. With Fix 1, recall drops over the first 10–20 words, because a short spam email shares its first piece with some of the added words, but then it stays almost flat: further words fill pieces of their own and no longer touch the spam piece. With 80 added words, Fix 1 still catches 0.601–0.629 of spam, against 0.532–0.590 with 20 words. The figure also shows the limit of the fix. Against a small amount of padding (5 or 10 words), Fix 1 is *worse* than the original model, because its higher threshold already misses more spam. It pays off once the attacker adds about 20 or more words (Naive Bayes: 20, Logistic Regression: 20, MLP: 40 words).

Discussion

RQ1. The MLP was the most accurate model, but only slightly ahead of Logistic Regression, and both clearly beat Naive Bayes. The reasons follow from how the models work. Naive Bayes assumes that words are independent. In email this is false, because

phrases such as "click here" or "unsubscribe from this list" contain many words that always occur together, so their evidence is counted several times and the predicted probabilities become overconfident and poorly ranked. Logistic Regression learns its weights jointly and so corrects for correlated words. The MLP can also combine words non-linearly. That bought a small gain in F1 and ROC-AUC, but at roughly 40 times the training cost, with weights that are much harder to interpret. TF-IDF already turns spam detection into a problem that is close to linearly separable, so the extra flexibility of the hidden layer has little left to add. On clean data alone, Logistic Regression offers the best balance of accuracy, speed and transparency, and the MLP is a reasonable choice when the last fraction of a percent matters and compute is not a constraint.

RQ2. All three models were robust to the character-level tricks often associated with spam (leet and spacing), but none was robust to padding with legitimate-looking words. This matches the findings of Wittel and Wu (2004) and Lowd and Meek (2005), almost twenty years later and with a neural model included. The more accurate models were the *less* robust ones. Logistic Regression and the MLP lost much more recall under padding than Naive Bayes, apparently because they rely heavily on a small set of very strong ham words. The answer to RQ1 therefore depends on the threat: the most accurate model on clean email is not the safest model against an attacker. The hidden layer of the MLP did not make it fundamentally more robust. It still sees the same bag-of-words input, and that representation lets an attacker change the balance of evidence simply by adding words. Retraining on disguised examples closes the specific holes it is shown, but it does not generalize to new variants. Robustness is therefore a property of the representation and the training data more than of the classifier.

Overfitting and underfitting. The learning curves show that Naive Bayes underfits, while Logistic Regression and the MLP have a small and shrinking generalization gap. Early

stopping was essential for the MLP: without it, validation loss rose after epoch 4 while training loss continued to fall. Regularization in Logistic Regression played the same role, with C = 10 performing better than both weaker and stronger penalties.

RQ3. Of the two fixes, only scoring in pieces gave protection that does not depend on the attacker's words. It works because it changes the unit the model looks at, so appended words can no longer dilute the spam text. The price is lower recall on normal spam at the same false-alarm rate, and the fix only pays off against larger amounts of padding; against a handful of added words, the original model is better. In practice it is therefore a trade-off a mail provider would tune. For example, a filter could use whole-email scoring for most mail and piece scoring as a second check. Retraining with padded spam, which is the most common advice, only taught the models to ignore the specific words they saw, because each word has its own weight. The general lesson of the project is that robustness has to come from how an email is represented or scored, not only from more training data.

Limitations

The study has several limitations. First, the data is old: the newest corpus is from 2008, while modern spam and phishing use different products, languages and formats, including images and links to cloud services. Second, much of the ham comes from a single company (Enron) and from a few open-source mailing lists, and Figure 10 shows that the models partly learned these sources instead of general properties of legitimate email. Third, the split is random. A time-based split, training on older emails and testing on newer ones, would be more realistic and would probably give lower scores. Fourth, only exact duplicates were removed, so near-duplicate spam from the same campaign can still appear in both training and test sets, which may make the scores slightly optimistic.

Fifth, the attacks were simple and fixed in advance. Real attackers adapt: an attacker who knows about piece scoring could spread innocent words through the whole email instead

of adding them at the end, so that every 50-word piece is diluted, and Fix 1 would not stop that. Sixth, the piece size of 50 words and the minimum of 25 words for the last piece were chosen once and not tuned. Finally, computation limited the neural model to a single hidden layer on TF-IDF features. Models that read word order, such as convolutional networks or pretrained transformers, were not tested.

Future Work

Several improvements follow from these limitations. A time-based evaluation (for example, training on TREC 2005 and testing on CEAS 2008) would measure how well each model handles the change in spam over time, and near-duplicate detection, for example with MinHash, would remove leakage between spam campaigns. For robustness, the piece size could be tuned, overlapping pieces could be used, and whole-email and piece scores could be combined so that the filter keeps its accuracy on normal spam while resisting padding. The fixes should also be tested against an adaptive attacker who mixes innocent words into the spam text or chooses them using the model's own weights. Finally, pretrained transformer models such as DistilBERT or BERT (Devlin et al., 2019) could be fine-tuned and compared with the linear models for both accuracy and robustness.

Conclusion

This project compared Multinomial Naive Bayes, Logistic Regression and a one-hidden-layer neural network for email spam detection on 122,850 unique emails from TREC 2005, CEAS 2008 and Enron-Spam, then tested how easily the models can be fooled and how the weakness can be reduced. Table 11 summarizes the three research questions, what was done, and the main results.

Table 11

Summary of the Project: Problem, Method, and Result

Question	What we did	Main result
RQ1: Which model is most accurate?	Trained Naive Bayes, Logistic Regression and an MLP on the same TF-IDF features; tuned on validation data; tested once on 18,428 unseen emails	MLP best (F1 0.9866); Logistic Regression almost equal (F1 0.9845) and about 40× faster to train; Naive Bayes weakest (F1 0.9667)
RQ2: Can the models be fooled?	Disguised the test spam: digits for letters, spaces inside words, and innocent words added at the end	Letter tricks barely matter; adding about 30 ham words cuts spam recall to 0.194–0.461 (combined attack)
First fix attempt	Retrained with disguised copies that used the attack's own word list	Protects only against the words it was trained on; other innocent words still work
RQ3, Fix 1	Score 50-word pieces; the email gets the highest piece score	Recall under the combined attack rises to 0.603–0.636, but recall on normal spam falls to 0.768–0.895 at the same false-alarm rate
RQ3, Fix 2	Train with spam plus random ham-typical words (none of the test words)	No gain against new words: combined-attack recall 0.157–0.414

On normal email all three models met the goals of the proposal. The MLP was the most accurate (F1 0.9866), Logistic Regression was almost as accurate while training about 40 times faster, and Naive Bayes was the fastest but least accurate. When the threshold is set for a very low false-alarm rate, Logistic Regression and the MLP are clearly better than Naive Bayes.

The disguised-spam experiment showed that high accuracy on clean data does not mean robustness. Changing letters inside spam words barely mattered, but adding a few dozen legitimate-looking words let most spam through all three models, and the most accurate models were fooled most often. Retraining with disguised spam protected only against the exact words it was trained on. Scoring each email in 50-word pieces was the only fix that worked against any added words, as long as the attacker adds about 20 words or more: it raised recall under the combined attack to 0.603–0.636, at the cost of missing more normal spam at the same false-alarm rate.

In practice, these results suggest that a TF-IDF model with Logistic Regression is a strong, cheap and interpretable baseline for a spam filter, with its threshold tuned for a low false-positive rate. A neural model adds a little accuracy but no real protection against attackers. A deployed filter should not rely on word content alone: it should score long emails in parts, be retrained regularly on new spam, and use signals that are harder to fake, such as sender reputation and link analysis.

References

Androutsopoulos, I., Koutsias, J., Chandrinos, K. V., & Spyropoulos, C. D. (2000). An experimental comparison of naive Bayesian and keyword-based anti-spam filtering with personal e-mail messages. In *Proceedings of the 23rd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 160–167). ACM. https://doi.org/10.1145/345508.345569

Biggio, B., & Roli, F. (2018). Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition, 84*, 317–331. https://doi.org/10.1016/j.patcog.2018.07.023

Champa, A. I., Rabbi, M. F., & Zibran, M. F. (2024). Curated datasets and feature analysis for phishing email detection with machine learning. In *Proceedings of the 2024 IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI)*. IEEE.

Cormack, G. V. (2008). Email spam filtering: A systematic review. *Foundations and Trends in Information Retrieval, 1*(4), 335–455. https://doi.org/10.1561/1500000006

Cormack, G. V., & Lynam, T. R. (2005). TREC 2005 spam track overview. In *Proceedings of the Fourteenth Text REtrieval Conference (TREC 2005)*. National Institute of Standards and Technology.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). https://doi.org/10.18653/v1/N19-1423

Drucker, H., Wu, D., & Vapnik, V. N. (1999). Support vector machines for spam categorization. *IEEE Transactions on Neural Networks, 10*(5), 1048–1054. https://doi.org/10.1109/72.788645

Guzella, T. S., & Caminhas, W. M. (2009). A review of machine learning approaches to spam filtering. *Expert Systems with Applications, 36*(7), 10206–10222. https://doi.org/10.1016/j.eswa.2009.02.037

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

K2509118. (2025). *Seven phishing/spam email datasets* [Data set]. Hugging Face. Retrieved November 9, 2026, from https://huggingface.co/datasets/K2509118/seven-phishing-email-datasets_pub

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*. https://arxiv.org/abs/1412.6980

Lowd, D., & Meek, C. (2005). Good word attacks on statistical spam filters. In *Proceedings of the Second Conference on Email and Anti-Spam (CEAS 2005)*.

McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). Spam filtering with naive Bayes – Which naive Bayes? In *Proceedings of the Third Conference on Email and Anti-Spam (CEAS 2006)*.

National Institute of Standards and Technology. (n.d.). *TREC spam track data* [Data set]. Retrieved November 9, 2026, from https://trec.nist.gov/data/spam.html

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Sahami, M., Dumais, S., Heckerman, D., & Horvitz, E. (1998). A Bayesian approach to filtering junk e-mail. In *Learning for Text Categorization: Papers from the 1998 Workshop* (AAAI Technical Report WS-98-05). AAAI Press.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

SetFit. (2022). *enron_spam* [Data set]. Hugging Face. Retrieved November 9, 2026, from https://huggingface.co/datasets/SetFit/enron_spam

Wittel, G. L., & Wu, S. F. (2004). On attacking statistical spam filters. In *Proceedings of the First Conference on Email and Anti-Spam (CEAS 2004)*.